

## 1. PRESENTATION DU PROJET

TaxiG est une plateforme complete de reservation de taxi en temps reel, inspiree d'Uber/Bolt. Elle connecte clients, chauffeurs et administrateurs via une interface web moderne.

### 1.1 Objectifs

- \* Permettre aux clients de commander un taxi en moins de 30 secondes
- \* Optimiser l'attribution des courses aux chauffeurs les plus proches
- \* Fournir un suivi GPS en temps reel
- \* Automatiser le calcul des tarifs
- \* Offrir des outils de gestion pour l'administration

### 1.2 Stack Technique

- \* Frontend: React.js 18 + ShadCN/UI + Tailwind CSS + Leaflet
- \* Backend: FastAPI (Python 3.11) + Motor (MongoDB async)
- \* Base de donnees: MongoDB Atlas (Cloud)
- \* Cartographie: OpenStreetMap + Google Maps API
- \* Authentification: JWT (JSON Web Tokens)
- \* Paiements: Stripe
- \* PDF Export: jsPDF + autoTable

## 2. ARCHITECTURE SYSTEME

### 2.1 Architecture 3-Tiers

L'application suit une architecture classique en 3 couches:

- \* PRESENTATION: React SPA (Single Page Application)
- \* APPLICATION: API REST FastAPI
- \* DONNEES: MongoDB Atlas Cluster

### 2.2 Flux de Communication

1. Le client fait une requete HTTP vers /api/\*
2. FastAPI valide les donnees avec Pydantic
3. Le serveur query MongoDB via Motor (async)
4. La reponse JSON est renvoyee au client
5. React met a jour l'interface

### 2.3 Structure des Fichiers

- \* /app/backend/server.py - API FastAPI principale
- \* /app/frontend/src/pages/client/ - Interface client
- \* /app/frontend/src/pages/chauffeur/ - Interface chauffeur
- \* /app/frontend/src/pages/admin/ - Interface admin
- \* /app/frontend/src/components/ui/ - Composants ShadCN

## 3. MODELE DE DONNEES

### 3.1 Collection 'clients'

- \* id: UUID v4 (identifiant unique)
- \* email: String (unique, pour connexion)
- \* hashed\_password: String (bcrypt)
- \* prenom, nom: String
- \* telephone: String
- \* created\_at: DateTime

### 3.2 Collection 'chauffeurs'

- \* id: UUID v4
- \* code\_chauffeur: String (ex: TAXI001, pour connexion)
- \* hashed\_password: String (bcrypt)
- \* prenom, nom, telephone, email: String
- \* vehicule: {marque, modele, immatriculation, couleur}
- \* is\_online: Boolean (en service ou non)
- \* position: {lat, lng} (GPS temps reel)

### 3.3 Collection 'course\_requests'

- \* id: UUID v4
- \* commande\_no: String (ex: TG-20260207-ABC123)
- \* client\_id, chauffeur\_id: UUID (relations)
- \* pickup\_lat, pickup\_lng, pickup\_address: Depart
- \* destination\_lat, destination\_lng, destination\_address: Arrivee
- \* distance\_km, duration\_minutes: Float
- \* prix\_estime, prix\_final: Float
- \* status: pending/assigned/in\_progress/completed/cancelled
- \* payment\_method: cash/card

## 4. ALGORITHME DE TARIFICATION

### 4.1 Formule de Calcul du Prix

$\text{PRIX\_TOTAL} = \text{TARIF\_BASE} + \text{TARIF\_DISTANCE} + \text{TARIF\_TRAFIC} + \text{TARIF\_ATTENTE}$

Ou:

- \*  $\text{TARIF\_BASE} = 6.30 \text{ EUR (prise en charge)}$
- \*  $\text{TARIF\_DISTANCE} = \text{distance\_km} \times 3.20 \text{ EUR/km}$
- \*  $\text{TARIF\_TRAFIC} = (\text{duree} \times 0.30) \times 0.50 \text{ EUR/min}$
- \*  $\text{TARIF\_ATTENTE} = \text{attente\_minutes} \times 0.70 \text{ EUR/min}$
- \*  $\text{COMMISSION\_TAXIG} = 15\% \text{ du prix total}$

### 4.2 Exemples de Calcul

EXEMPLE 1 - Course courte (3 km, 8 min):

Base: 6.30 + Distance: 9.60 + Trafic: 1.20 = 17.10 EUR

EXEMPLE 2 - Course moyenne (5.8 km, 15 min, 2 min attente):

Base: 6.30 + Distance: 18.56 + Trafic: 2.25 + Attente: 1.40 = 28.51 EUR

EXEMPLE 3 - Course longue (150 km, 105 min):

Base: 6.30 + Distance: 480.00 + Trafic: 15.75 = 502.05 EUR

## 5. ALGORITHME DE DISPATCH (CHAUFFEUR LE PLUS PROCHE)

### 5.1 Principe

Le systeme attribue automatiquement chaque course au chauffeur disponible le plus proche du point de prise en charge, en utilisant la formule de Haversine.

### 5.2 Formule de Haversine

La formule de Haversine calcule la distance entre deux points GPS sur une sphere (la Terre).

Donnees d'entree:

- \* Point A (client): lat1, lon1 en degres
- \* Point B (chauffeur): lat2, lon2 en degres
- \* R = 6371 km (rayon de la Terre)

Etapas du calcul:

- \* 1. Convertir les coordonnees en radians
- \* 2. Calculer les differences: delta\_lat, delta\_lon
- \* 3. Appliquer:  $a = \sin^2(dlat/2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(dlon/2)$
- \* 4. Calculer:  $c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$
- \* 5. Distance = R \* c (en km)

### 5.3 Algorithme de Selection

1. Recuperer tous les chauffeurs en ligne (is\_online=true)
2. Filtrer ceux avec une position GPS valide
3. Calculer la distance de chaque chauffeur au client
4. Trier par distance croissante
5. Selectionner le premier (le plus proche)
6. Estimer l'ETA (distance / 30 km/h en ville)

## 6. FLUX FONCTIONNELS

### 6.1 Cycle de Vie d'une Course

PENDING -> ASSIGNED -> IN\_PROGRESS -> COMPLETED

- \* PENDING: Course creee, en attente d'attribution
- \* ASSIGNED: Chauffeur assigne, en route vers client
- \* IN\_PROGRESS: Client recupere, en route vers destination
- \* COMPLETED: Arrivee a destination, paiement effectue
- \* CANCELLED: Annulee par client ou chauffeur

### 6.2 Flux de Reservation Client

- \* 1. Client ouvre l'app et autorise la geolocalisation
- \* 2. Carte affiche sa position et les taxis disponibles
- \* 3. Client saisit la destination (autocomplete Google)
- \* 4. Systeme calcule distance, duree et prix estime
- \* 5. Client confirme et choisit le mode de paiement
- \* 6. Dispatch trouve le chauffeur le plus proche
- \* 7. Chauffeur recoit notification et accepte
- \* 8. Client suit le chauffeur en temps reel
- \* 9. Course terminee, paiement effectue

### 6.3 Flux Chauffeur

- \* 1. Connexion avec code chauffeur + mot de passe
- \* 2. Activation du service (bouton En ligne)
- \* 3. Position GPS envoyee au serveur (toutes les 3s)
- \* 4. Reception notification nouvelle course
- \* 5. Acceptation ou refus de la course
- \* 6. Navigation GPS vers le client
- \* 7. Demarrage de la course (chrono actif)
- \* 8. Navigation vers la destination
- \* 9. Fin de course, affichage prix final

## 7. API REST - ENDPOINTS PRINCIPAUX

### 7.1 Authentification

- \* POST /api/client/register - Inscription client
- \* POST /api/client/login - Connexion client
- \* POST /api/chauffeur/login - Connexion chauffeur
- \* POST /api/admin/login - Connexion admin

### 7.2 Gestion des Courses

- \* POST /api/course/estimate - Estimation du prix
- \* POST /api/course/create - Creation d'une course
- \* GET /api/course/{id} - Details d'une course
- \* POST /api/course/{id}/cancel - Annulation

### 7.3 Operations Chauffeur

- \* GET /api/chauffeur/profile - Profil du chauffeur
- \* POST /api/chauffeur/status - Changer statut en ligne/hors ligne
- \* POST /api/chauffeur/position - Mise a jour GPS
- \* GET /api/chauffeur/pending-course - Course en attente
- \* POST /api/chauffeur/course/{id}/respond - Accepter/Refuser
- \* POST /api/chauffeur/course/{id}/start - Demarrer course
- \* POST /api/chauffeur/course/{id}/complete - Terminer course
- \* GET /api/chauffeur/revenus - Statistiques revenus

### 7.4 Administration

- \* GET /api/admin/stats - Statistiques globales
- \* GET /api/admin/chauffeurs - Liste des chauffeurs
- \* POST /api/admin/chauffeurs - Creer un chauffeur
- \* GET /api/admin/courses - Toutes les courses

## 8. SECURITE

### 8.1 Authentification JWT

Les tokens JWT sont generes a la connexion et valides pendant 24h.

- \* Algorithme: HS256
- \* Payload: {user\_id, role, exp}
- \* Header Authorization: Bearer <token>

### 8.2 Hashage des Mots de Passe

Les mots de passe sont hashes avec bcrypt (12 rounds).

- \* Jamais stockes en clair
- \* Verification securisee avec timing constant

### 8.3 Protection des Routes

- \* Middleware d'authentification sur toutes les routes sensibles
- \* Verification du role (client/chauffeur/admin)
- \* Validation des donnees entrantes avec Pydantic
- \* CORS configure pour les origines autorisees

## 9. IDENTIFIANTS DE TEST

### 9.1 Comptes de Test

Client:

- \* Email: jean.dupont@test.com
- \* Mot de passe: test123

Chauffeur Principal:

- \* Code: TAXI001
- \* Mot de passe: chauffeur123

Chauffeurs de Test (Geneve):

- \* Codes: GVA001 a GVA040
- \* Mot de passe: test123

Administrateur:

- \* Username: naim
- \* Mot de passe: admin123

### 9.2 URL de l'Application

<https://taxig-debug.preview.emergentagent.com>